PiPER 视觉抓取 · 简明流程

目标：相机识别桌上方盒 → 机械臂抓取。固件 S-V1.9-0，2026-11-04 实测。

1 坐标链

```
像素(u,v)+深度 d --内参 K⟶ 相机系 --T_flange→cam⟶ 法兰 --正运动学⟶ 基座 ⟶ move_p
```

`T_flange→cam` 用手眼标定（AX=XB）求，唯一门槛。

2 硬件与自检

- PiPER 六轴 + 平行夹爪，CAN 1 Mbps，反馈 200 Hz
- 相机 Orbbec Dabai DC1，装夹爪上方（eye-in-hand），倒装 → 取图必须 `--rotate 180`
- 限位(度)：J1 ±150 J2 0~180 J3 −170~0 J4 ±100 J5 ±69.9 J6 ±120；夹爪 0~70 mm

```
lsusb | grep -E "1d50|2bc5"

ip -details link show can0     # UP, 1000000

systemctl status can0-piper
```

3 环境

- `/home/po/arm-env` py3.13：pyAgxArm。S-V1.9-0 只能用新 SDK，piper_sdk 仅适配 ≤S-V1.8-8
- `/home/po/vision-env` py3.11：pyorbbecsdk 源码编译。PyPI wheel 是 macOS 二进制；分支必须 `main`（v1.10.16），`v2-main` 不支持 DC1

4 控制

接口：`get_joint_angles` `get_flange_pose` `get_arm_status` `enable` `set_speed_percent` `move_j` `move_p` `move_gripper_m`

坑： 1. 单帧指令无效，须连发 0.8 s（≥20 帧） 2. 首帧丢：上电 `ctrl_mode=TEACHING(0×2)`，首条指令切 `CAN_CTRL(0×1)`，该帧被弃 3. J5 增大 = 视角下压（实测 16.55°→41.64° 正对桌面）。先 ±10° 试方向 4. `calibrate_joint` 改零点，勿调；`disable`/`reset` 掉臂，先托住

5 视觉

```
/home/po/vision-env/bin/python /home/po/vision/capture.py --rotate 180 --tag t1
/home/po/vision-env/bin/python /home/po/vision/detect_box.py --tag t1
```

输出 `out/<tag>_{color.png, depth.npy(已对齐彩色, mm), depth_vis.png, detect.png}`

- 深度 0.3~1.5 m 有效；反光/透明/纯黑失效
- SDK 内参 `cy=214.66` 可疑（640×480 应为 240）→ 必须自标
- 检测 = HSV 棕色 + 深度门限 + 连通域面积/长宽比/实心度 + minAreaRect

当前：J5=41.64°，`P_cam=(75.6, -21.1, 409.0) mm`，102×63 mm，yaw 0°，solidity 0.86

6 标定

靶：黑白棋盘格 9×6、方格 25 mm。用 `calib/chessboard_9×6_25mm.pdf`（A4 实际大小）或平板全屏 `chessboard_tablet_9×6_25mm.png`。切割垫不行（非黑白方格）。打印后量一格，实测值传 `--square`。靶全程不动。

```
# 内参：多角度拍 15~20 张，判据 rms < 0.5 px
/home/po/vision-env/bin/python /home/po/vision/calibrate_intrinsic.py

# 手眼：臂自动走 16 位姿（速度 8%），每位姿拍图 + 读法兰
/home/po/arm-env/bin/python /home/po/vision/handeye.py collect --poses 15 --square 25
/home/po/arm-env/bin/python /home/po/vision/handeye.py solve   --square 25
```

判据：4 种算法（TSAI/PARK/HORAUD/ANDREFF）平移互差 < 10 mm，量级几十 mm。 失败原因：靶被碰 / 位姿无转动退化 / 过曝。

验证：走到盒上方 80 mm 停住，人眼确认再下降。

7 抓取

```
P_cam   = ((u-cx)d/fx, (v-cy)d/fy, d)          d = 轮廓内深度中值
P_base  = T_base→flange(拍照瞬间) · T_flange→cam · P_cam
```

```
approach= [Px, Py, Pz+80]
grasp   = [Px, Py, Pz+盒高/2]                  夹爪开度 = 盒宽-10 mm
```

序列（速度 10%，每步发 0.8 s 并等 `motion_status=0`）：

1. 张开夹爪
2. `move_p` → approach
3. `move_p` → grasp
4. 闭合 1~2 N，停 0.3 s，看开度反馈确认夹住
5. 抬 80 mm

相机在夹爪上方，下降末段被夹爪遮挡 → 开环，靠 approach 定准。

8 安全

- 无抱闸，断电掉臂：`disable`/`reset`/拔线前托住末端
- 首次运动 10~15% 速度、5~10°
- Z ⩾ 20 mm；SDK 只拦关节限位，不拦碰撞
- 通电不热插拔 CAN
- 急停按物理按钮，`arm.py estop` 是软件急停（事后重新 enable）

9 排错

- can0 不存在/DOWN：USB-CAN 未插好 → `sudo /home/po/arm/can_up.sh`
- 发指令不动：单帧/首帧丢 → 连发 0.8 s，确认 `ctrl_mode=0×1`
- 动一下报错：逆解失败 → 换中间位姿
- 相机打不开：udev 未生效 → 重插，`/dev/dabai_dc1*` 应 0666
- 深度大片 0：太远/反光 → 目标移 0.3~1.2 m
- 检测抓到背景：视角太平 → 先压腕俯视
- 棋盘格检不出：靶非黑白方格 → 换标准棋盘格
- 手眼四算法差大：位姿旋转轴单一 → 加绕不同轴转动重采

10 文件与进度

```
/home/po/arm/     arm.py  can_up.sh  README.md
/home/po/vision/  capture.py  detect_box.py  arm_move.py
                  calibrate_intrinsic.py  handeye.py
                  calib/  (棋盘格, handeye_data.npz)
                  out/    (图 + 内参/手眼 .npz)
```

已完成：硬件链路、控制脚本、取图、方盒检测、视角压到 41.64°、标定脚本

待完成：棋盘格靶 → 内参 → 手眼 → approach 预演 → 抓取

11 自动化：不调用 LLM 也能跑

结论：**这条流程不需要任何神经网络，也不需要 LLM。** 感知是颜色+深度的经典 CV，几何是标定死的常数，动作是固定状态机。

11.1 为什么不需要「模型」

- **识别**：棕色 HSV 阈值 + 深度门限 + 连通域过滤 = 几行 OpenCV。盒子这种高对比目标不需要 YOLO。
- **定位**：不用手眼矩阵（本机那套不可靠），用**视觉伺服**闭环——把盒子质心挪到示教像素。
- **高度**：不用深度换算，抓取高度是示教常数（法兰 z=139mm）。
- **LLM 的历史作用**：试错、发现「夹爪单帧命令无效」「指长 158 是错的」「开口能到 78 不是 70」这些坑，并把它们固化成参数。流程跑通后 LLM 就是多余的。

11.2 单脚本 `autograsp.py`（239 行）

```
autograsp.py --calib     # 一次性: 测 像素→基座 的 2x2 雅可比, 存 calib/servo_calib.json
autograsp.py             # 抓取: 观察位姿 → 视觉伺服对中 → 10mm 分段降到 139 → 闭合 → 抬起
autograsp.py --release   # 放回: 降到 139 → 张开 → 抬走
autograsp.py --home      # 归位: 关节空间 3 段回零 → 法兰 (56,0,213) mm
```

流程：

1. 连 CAN，夹爪连发帧张开到 **78mm**

2. 走观察位姿（法兰 189mm，视野里能看全盒子）
3. 取图 → 棕色连通域 → 质心 (u,v) + 深度
4. 视觉伺服：`Δ基座 = Jinv · (目标像素 − 当前像素)`，单次 ≤15mm，≤6 次
5. 分段下降：每段 10mm，**每段用 FK 校验到位误差**，>2mm 立即抬回退出
6. 闭合到 58mm / 3N（连发 2s），读回宽度和力
7. **判定**：宽度 > 60mm 且 |力| > 0.5N 才算夹住，否则张开退出
8. 抬起 50mm 再读一次宽度/力，验证没掉

11.3 换对象/换位置要改什么

情况	要动的参数
盒子换颜色	`find_box()` 的 HSV 阈值
盒子换位置	不用改（伺服负责对中），只要还在臂展内
盒子换大小/高度	`grasp_z`、`close_mm`
相机挪过	重跑 `--calib` 并重测 `target_px`
要抓任意物体	`find_box()` 换成 YOLOv8n（6MB，CPU 30ms），其余不变

11.4 关键常量（本机实测）

```
obs_z      189.0 mm          对中/观察高度
grasp_z    139.0 mm          抓取高度（指尖落在盒子上半部）
target_px  (382.2, 281.5)    盒子正对夹爪时的质心（相机不在正中心，明显偏右）
open_mm    78.0              实测可开（说明书只写 70）
close_mm   58.0 @3N          停在实际 66mm / 2.1N 即为夹住
speed      5 %
```

11.5 实跑记录（2026-11-05，端到端成功 ×2）

```
autograsp.py --calib   →  J(px/mm) = [[3.30,-0.11],[-0.16,-3.34]]
```

```
autograsp.py            →  伺服 2 次(末误差 0.6px) → 5 段下降(每段误差≤0.5mm)
                           → 闭合 65.9mm/-2.09N → 抬 50mm 仍 65.8mm → 抓取成功
autograsp.py --release →  降到 139 张开 77.6mm 抬走，盒子回原位
```

11.6 三个坑（2026-11-05 发现）

- **近距离深度整帧为 0**：DC1 在约 150mm 以内深度全 0（观察高度 189mm 正是如此）。`find_box()` 因此改成「深度有效才用深度门限，否则纯颜色」；`capture.py` 加了暖机（连抓帧直到有效深度达 10%）与空深度容错，不再抛 `no frames`。
- **白标签把质心拉偏**：纸箱那张白色贴纸没有红外回波，深度是 0；原来的深度门限 `80<d<400` 会把这整块切掉，只留下盒子下缘 → 质心被拉低约 100px → 伺服对着一个假位置来回摆。现在深度门限只用来剔远景：`(d < 400) | (d == 0)`。
- **像素增益随距离变**：示教处 `3.3 px/mm`，离得远时实测只有 `1.5~2`；固定增益单步 15mm 会过冲。加线搜索：误差不降就把步长减半（15→8→4→3mm）。修后从 180px 偏差 **6 步收敛到 1.7px**。

11.7 盒子摆歪了也能抓（角度伺服，2026-11-05 下午）

- 目标：让「短边」在画面里水平 —— 夹爪沿画面 x 开合，只有短边水平才夹在 61mm 那侧（开口 78mm，夹长边 112mm 必夹空）。
- 做法：只转 J6。先试探 12° **现场实测增益**（minAreaRect 的角度约定会随位姿反号，标定值不可靠），再按实测增益闭环，步长 ≤20°，容差 3°。
- 转腕会让相机绕工具轴公转、画面平移：所以流程是「对中 → 转角度 → 再对中」交替收敛，主流程已内置两轮。
- 像素雅可比**随 J6 旋转**：align() 按 theta = yaw_gain × (J6 - calib_j6) 把误差转回标定朝向（Rm = R(-theta)），否则腕子转过 90° 后伺服会绕圈发散。
- J6 限位 ±115°：越限直接停止（打印「J6 将到 xxx 度, 越限, 停止」），不硬顶。
- 调试免摆盒子：--yaw-offset 30 对中后先把 J6 转 30°，等价于盒子被转了 30°。
- 观察高度从 189 抬到 228mm（法兰 z）：盒子居中后相机离盒顶仅 ~135mm，画面被撑满（403px 宽）；抬高后约 260px，尺寸与深度都更稳。
- 实测：--yaw-offset -45 全链成功（转角 2 步收敛 → 再对中 2 步 → 下降 9 段 → 闭合 65.6mm/-2.08N → 抬起保持）；无偏移基线同样成功。

- **大范围移动必须走关节空间**：`move_p` 从 home 直接去观察位姿会过腕部奇异区（昨天的砸机事故）。`goto_q()` 按 3 段关节插值（每段 ≤32°），`obs_q` / `home_q` 是示教关节位姿。